

《无题》

2025 普通高中不务正业水平选择性考试适应性演练

译码与执行能力

总分：150

试卷类型：A

考试时间：120 分钟

命题人：xtex (Zerozaki Ori)

审题人：xtex (Zerozaki Ori)

绝密 ★ 启用前

注意事项

1. 答卷前，考生务必将自己的姓名、考生号、考场号和座位号填写答题卡上，用 2B 铅笔将试卷类型 (A) 填涂在答题卡相应位置上，将条形码横贴在答题卡右上角“条形码粘贴处”。
2. 作答选择题时，选出每小题答案后，用 2B 铅笔在答题卡上对应题目选项的答案信息点涂黑；如需改动，用橡皮擦干净后，再选涂其他答案。答案不能答在试卷上。写在试卷、草稿纸和答题卡上的非答题区域均无效。
3. 非选择题必须用黑色字迹的钢笔或签字笔作答，答案必须写在答题卡各题目指定区域内相应位置上；如需改动，先划掉原来的答案，然后再写上新答案；不准使用铅笔和涂改液。不按以上要求作答无效。
4. 考生必须保持答题卡的整洁。考试结束后，将试卷和答题卡一并交回。
5. 试题中所有 x86-64 指令助记符及汇编采用 Intel 汇编形式，涉及推测译码或执行结果的，如无特殊声明，假定执行环境为长模式，CR 寄存器的 VMX 标志被关闭，通用寄存器、浮点寄存器和向量寄存器均已清零，CPL 为 Level 0，代码段选择子、数据段选择子和栈段选择子 RPL 为 Level 0，对应的 DPL 也为 Level 0，代码段、数据段、栈段指向相同的内存空间，数据段、栈段可读可写，SSE 2、SSE 3、AVX-512 和 VMX 扩展指令集可用且如果控制寄存器中存在对应的开关，则开关已被打开。
6. 试题中所有的 LoongArch 指令助记符及汇编采用官方手册形式，涉及推测译码或执行结果的，如无特殊声明，假定执行环境为 PLV0、页表映射地址翻译模式，\$r1 至 \$r31、浮点寄存器和向量寄存器均已清空，LoongArch64、LSX、LASX 指令集可用。
7. 试题中涉及推测译码或执行结果的，如无特殊声明，假定处理器为多核处理器，若操作系统存在，则操作系统识别并利用多核处理器。
8. 试题中所有“字节”均指 8 位元组。

第一部分 架构理解

第一节 x86-64（共 5 小题；每小题 3 分，满分 15 分）

阅读题目，从每题所给的 A、B、C、D 四个选项中，选出最佳答案。

1. x86-64 架构下，以下哪个指令可以在一条指令内将一个 64 位通用寄存器的值复制到另一个 64 位通用寄存器中？（ ）
A. MOV
B. NOP

- C. IN
- D. VCVTSH2SD
- 2. x86-64 架构下，以下哪个指令是非特权序列化指令？（ ）
 - A. MOV
 - B. SFENCE
 - C. CPUID
 - D. LGDT
- 3. x86-64 架构下，以下描述正确的是？（ ）
 - A. Linux 下，所有进程都可以稳定地通过连续两次执行非特权指令 RDTSC 读取 TSC，并将其值相减以测定时间。
 - B. Linux 下，若 KVM 已被成功启用，VirtualBox 仍可以同时使用。
 - C. 欲将 32 位通用寄存器 EAX 清零，MOV EAX, 0 是编码最短的方案。
 - D. 分页启用后，MMU 仍会在地址转换时考虑分段。
- 4. x86-64 架构下，控制保护模式开关的控制位和页目录基地址分别位于哪两个控制寄存器中？（ ）
 - A. CR0、CR1
 - B. CR0、CR3
 - C. CR1、CR1
 - D. CR4、CR3
- 5. x86-64 架构下，下列指令中，完全不会修改 EFLAGS 的是？（ ）
 - A. JMP
 - B. TEST
 - C. AND
 - D. CMP

第二节 LoongArch64（共 5 小题；每小题 3 分，满分 15 分）

阅读题目，从每题所给的 A、B、C、D 四个选项中，选出最佳答案。

- 1. 龙芯架构中，每条指令编码后的长度是（ ）
 - A. 1 字节
 - B. 4 字节
 - C. 8 字节
 - D. 因指令而异
- 2. 访存地址“自然对齐”指的是（ ）
 - A. 对齐到 1 字节
 - B. 对齐到 4 字节
 - C. 对齐到访存长度
 - D. 对齐到机器字长
- 3. 龙芯架构中，取值访存要求地址（ ）
 - A. 自然对齐
 - B. 对齐到访存长度
 - C. 对齐到机器字长
 - D. 对齐到 4 字节

4. 龙芯架构中，下列说法正确的是（ ）
- A. LASX 是龙芯架构的变长指令集扩展。
 - B. 不论 CSR 值如何，PLV3 始终不能通过 CSRRD 读取性能计数器。
 - C. 访存地址需要自然对齐，转移指令的目标地址也需要对齐。
 - D. 访存地址需要自然对齐，转移指令的目标地址不需要对齐。
5. 龙芯架构中，下列说法不正确的是（ ）
- A. r0 寄存器为零寄存器是 ELF ABI 的规定，故可通过 `ADD.D %r0, %r0, 1` 修改其值。
 - B. 龙芯架构是一款由龙芯研发的精简指令集架构。
 - C. 龙芯架构只采用小尾端的存储方式。
 - D. 龙芯架构中，通用寄存器 r3 被用作栈指针寄存器。

第二部分 解码

第一节 x86-64（共 5 小题；每小题 5 分，满分 25 分）

阅读机器码，从每题所给的 A、B、C、D 四个选项中，选出正确的汇编指令。

1. `B8 00 00 00 00`（ ）
- A. `MOV EAX, 0h`
 - B. `NOP`
 - C. `CMP EAX, 0h`
 - D. `INT 0h`
2. `EB 00`（ ）
- A. `INT 0h`
 - B. `JMP 0`
 - C. `JMP 2`
 - D. `JMP EAX`
3. `0F 01 C2`（ ）
- A. `ENCLV`
 - B. `VMXOFF`
 - C. `VMXON`
 - D. `VMLAUNCH`
4. `F3 FF C8`（ ）
- A. `DEC EAX`
 - B. `SUB EAX, 1`
 - C. `XOR EAX, C8`
 - D. `REPZ DEC EAX`
5. `B8 0A 00 00 00`（ ）
- A. `MOV EAX, 0A000000h`
 - B. `SUB EAX, 0A000000h`
 - C. `MOV EAX, 0Ah`
 - D. `SUB EAX, 0Ah`

第二节 LoongArch64（共 5 小题；每小题 5 分，满分 25 分）

阅读机器码，从每题所给的 A、B、C、D 四个选项中，选出正确的汇编指令。

- 03 00 80 03 ()
A. ST.D \$r3, \$r3, 0
B. LD.D \$r3, \$r3, 0
C. ORI \$r3, \$r0, 0
D. ANDI \$r3, \$r0, 0
- 00 00 2A 00 ()
A. B 0
B. SYSCALL 0
C. BREAK 0
D. DBAR 0
- A0 31 C0 2A ()
A. LD.D \$r0, \$t3, 12
B. LD.D \$r0, \$t1, 12
C. PRELD 0, \$t1, 12
D. PRELDX 0, \$t1, \$r12
- 21 04 57 38 ()
A. SC.Q \$ra, \$ra, \$ra
B. ORI \$r0, \$r0, 0
C. SC.D \$ra, \$ra, 1
D. ST.D \$ra, \$ra, 1
- 21 48 1e 01 ()
A. FRINT.S \$r1, \$r1
B. FRINT.D \$r1, \$r1
C. FRINT.S \$f1, \$f1
D. FRINT.D \$f1, \$f1

第三部分 执行

第一节 x86-64（共 5 小题；每小题 5 分，满分 25 分）

阅读汇编代码，从每题所给的 A、B、C、D 四个选项中，选出符合要求的结果描述。

- MOV EAX, 10
REPZ DEC EAX
代码运行结果描述正确的是 ()
A. EAX 为 0。
B. EAX 为 1。
C. EAX 为 10。
D. 产生 #DIV0 异常。
- CLI
INT 3
STI

代码运行结果描述正确的是 ()

- A. 由于 CLI, 不会触发中断。
- B. 在 INT 3 处触发 3 号中断。
- C. 在 STI 处触发 3 号中断。
- D. 产生 #GP 异常。

3. **JMP 0**

代码运行结果描述正确的是 ()

- A. 产生 #GP 异常。
- B. 产生 #UD 异常。
- C. 产生死循环。
- D. 等效于无操作。

4. `; XMM2 = 0xFFFF000000000000`

`; 其中, [4096-8191]为一个有效页面, [8192-12287]是无效页面。`

`VMASKMOVPS XMM1, XMM2, 8190`

代码运行结果描述正确的是 ()

- A. 由于缺页而等效于无操作。
- B. 由于缺页而产生 #GP 异常。
- C. 将 8190-8191 的值左移 112 位存入 XMM1, 低 112 位未修改。
- D. 将 8190-8191 的值左移 112 位存入 XMM1, 低 112 位清零。

5. `0x1000: MOV BYTE [0x1007], 0xCC`

`0x1007: NOP`

代码运行结果描述正确的是 ()

- A. 内存 0x1007 处被修改为 0xCC, 此外无操作。
- B. 内存 0x1007 处被修改为 0xCC, 但不一定触发 3 号中断。
- C. 内存 0x1007 处被修改为 0xCC, 并一定触发 3 号中断。
- D. 由于代码修改自身而触发 #GP 中断。

第二节 LoongArch64 (共 3 小题; 每小题 5 分, 满分 15 分)

阅读汇编代码, 从每题所给的 A、B、C、D 四个选项中, 选出符合要求的结果描述。

1. `ORI $r0, $r0, 1`

代码运行结果描述正确的是 ()

- A. 等效于无操作。
- B. r0 的值为 1。
- C. r0 的值未定义。
- D. 产生异常中断。

2. `B 0`

代码运行结果描述正确的是 ()

- A. 等效于无操作。
- B. 产生死循环。
- C. 该汇编代码无效。
- D. 产生异常中断。

3. B 1

代码运行结果描述正确的是 ()

- A. 无副作用。
- B. 由于目标地址为立即数左移 2 位后加上 PC，等效于无操作。
- C. 该汇编代码无效。
- D. 由于目标地址未对齐，产生异常中断。

第四部分 作文 (30 分)

阅读文本，补全两段 x86-64 汇编，选好角度、确定文意、不得套作、不得抄袭、使用 Intel 汇编语法、不得使用特权指令、不得使用虚拟化指令、向量指令或浮点指令、可使用 Linux 系统调用、可使用 nasm 预处理语法、内容积极向上，30 指令左右。

```
org 0x7c00
bits 32

main:
    jmp decrypt

flag: dd 12345
v0: dd 4984
v1: dd 132456
sum: dd 0
password: dd 0xB95ECB53, 0x2FBF6EF4, 0xBA6ACC36, 0xA8C730E8, 0xFC8780F7,
0x6296224, 0xAA79B131, 0x3C151A79
%define k0 0x1dbb6bf5
%define k1 0xb00d45c5
%define k2 0xb12c0fee
%define k3 0x8309e9ff
%define DELTA 0x9E3779B9

decrypt:
    mov [flag], eax
    mov eax, 4
    mov ebx, password
    .chunk:
        ; load data
        mov ecx, [ebx]
        mov [v0], ecx
        mov ecx, [ebx + 4]
        mov [v1], ecx
        pushad
        call decipher
        popad
        ; cmp
        mov ecx, [v0]
        mov edx, dword [flag]
        mov edx, [edx]
        cmp edx, ecx
        jne end
```

```

mov ecx, [v1]
mov edx, dword [flag]
mov edx, [edx + 4]
cmp edx, ecx
jne end
add dword [flag], 8

add ebx, 8
dec eax
cmp eax, 0
jne .chunk
end:
dyn_insn:
db 0xCC, 10
db 0x12, 0x13, 0x14, 0x15, 0x13, 0x12, 0x00, 0x11, 0x45, 0x14 ; rand

int 3

decipher:
mov dword [sum], 0xC6EF3720
mov eax, 32
.round:
pushad
call decipher_round
popad
dec eax
cmp eax, 0
jne .round
ret

decipher_round:
; eax = ((v0 << 4) + k2)
mov eax, [v0]
shl eax, 4
add eax, k2
; ebx = (v0 + sum)
mov ebx, [v0]
add ebx, [sum]
; ecx = ((v0 >> 5) + k3)
mov ecx, [v0]
shr ecx, 5
add ecx, k3
; eax = eax ^ ebx ^ ecx
xor eax, ebx
xor eax, ecx
; v1 -= eax

sub [v1], eax

```

```

; eax = eax ^ ebx ^ ecx
xor eax, ebx
xor eax, ecx

```

```
mov byte [dyn_insn], 0xeb ; JMP rel8

```

```

; v0 -= eax
sub [v0], eax
; sum -= DELTA
sub dword [sum], DELTA
ret

```